

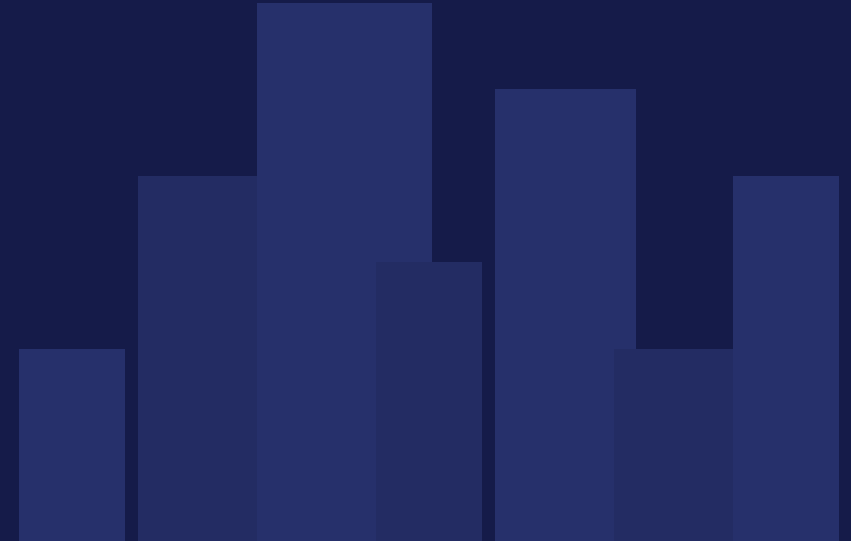
SQL ANALYTICS PROJECT

Real Estate Market Analytics

20 Advanced KPIs across Residential Listings, Rental Market & Transaction Trends — built with Window Functions, CTEs, Pivots & Statistical SQL

Dataset Scope: 7,000 Listings • 4,500 Rentals • 4,800 Transactions • 20 Cities

SQL Server • Window Functions • CTEs • Pivot • Statistical Analysis



Turning Raw Property Data into Decision-Ready Insight

This project simulates a real-world real estate intelligence engine: 20 SQL KPIs spanning ranking, time-series trend, pivoting, forecasting, and outlier detection — answering the exact questions buyers, landlords, investors and analysts ask.



Pricing Intelligence

Rank cities & localities by price/sqft; segment listings into price bands.



Trend & Forecasting

Quarter-over-quarter trend lines plus a 5-year recursive price projection.



Investment Analytics

Gross rental yield ranking and statistical (z-score) deal-quality screening.



Risk & Quality Flags

Auto-flag high-cost-burden deals and stale, slow-moving listings.

Three Linked Datasets, One Unified Market View



dbo.residential_listings

7,000 rows

listing_id, city, locality, property_type, total_price_lakh, price_per_sqft, days_on_market, builder_name...



dbo.rental_market

4,500 rows

rental_id, city, monthly_rent_inr, annual_rent_inr, rent_per_sqft, deposit, tenant_type...



dbo.transactions_trends

4,800 rows

transaction_id, sale_price_inr, price_per_sqft, sale_year/quarter, cap_rate_percent, yoy_price_appreciation_pct...

Ranking Cities & Localities by Price per Sq.Ft.

KPI 1 — RANK()

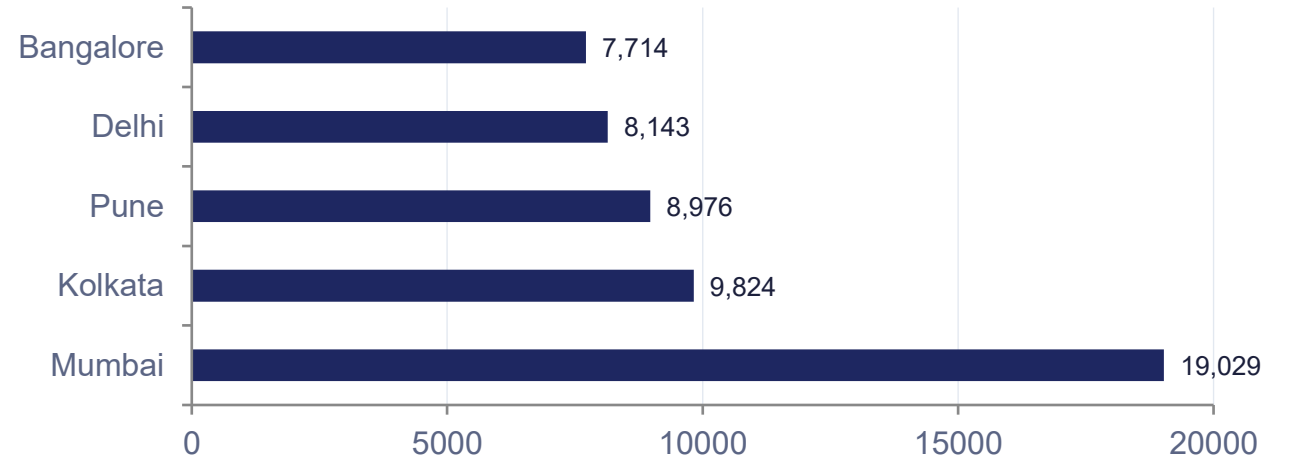
A buyer wants to know how each city ranks by average price/sqft, citywide.

KPI 2 — DENSE_RANK()

Within each city, localities are ranked to spot premium micro-pockets, with no rank gaps on ties.

```
SELECT city,  
       AVG(price_per_sqft) AS avg_price_sqft,  
       RANK() OVER (  
         ORDER BY AVG(price_per_sqft) DESC  
       ) AS city_price_rank  
FROM dbo.residential_listings  
GROUP BY city;
```

Avg. Price / Sq.Ft by City — Top 5 (₹)



Insight: Mumbai commands a 2x premium over the #2 ranked city (Kolkata), confirming its position as the outright luxury benchmark market.

Segmenting Listings into Price Quartiles

KPI 3 — NTILE(4)

All 7,000 listings (₹30L – ₹667L) are split into 4 equal-sized price bands — from Budget to Luxury — to drive targeted marketing campaigns.

```
SELECT listing_id, city, total_price_lakh,  
       NTILE(4) OVER (  
         ORDER BY total_price_lakh  
       ) AS price_quartile  
FROM dbo.residential_listings;
```

Q1 • Budget

₹30L – ₹118L

1,750 listings

Q2 • Mid-Range

₹118L – ₹207L

1,750 listings

Q3 • Premium

₹207L – ₹330L

1,750 listings

Q4 • Luxury

₹330L – ₹667L

1,750 listings

Equal-frequency banding gives marketing exactly four evenly sized segments to target — instead of arbitrary price cut-offs.

Quarter-over-Quarter Price Momentum

KPI 4 — LAG()

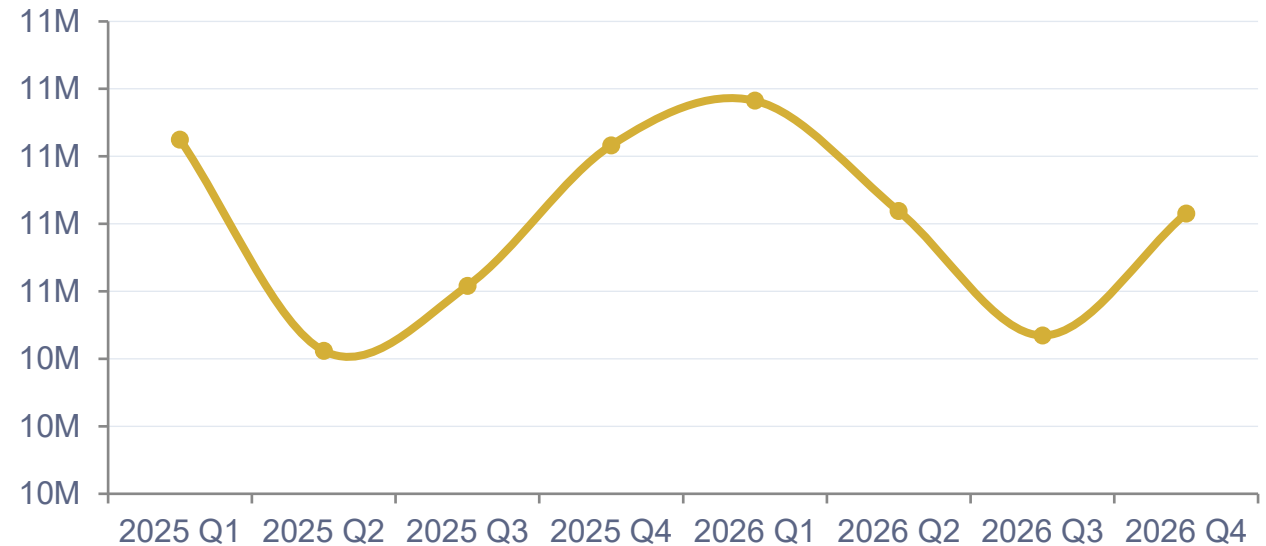
Compares each quarter's avg sale price to the prior quarter to flag slowdowns.

KPI 5 — LEAD()

Compares current price/sqft to the next quarter's value to validate forecast accuracy.

```
SELECT sale_year, sale_quarter,  
       AVG(sale_price_inr) AS avg_sale_price,  
       LAG(AVG(sale_price_inr)) OVER (  
         ORDER BY sale_year, sale_quarter  
       ) AS prev_qtr_price  
FROM dbo.transactions_trends  
GROUP BY sale_year, sale_quarter;
```

Avg. Sale Price by Quarter (₹, last 8 quarters)



Insight: Sale prices oscillate within a tight $\pm 6\%$ band each quarter — a mature, stable market rather than one trending sharply up or down.

Spotting the Cheapest & Costliest Listing per City



KPI 6 — FIRST_VALUE()

Returns the lowest-priced active listing in every city — instant entry-level leads for investors.



KPI 7 — LAST_VALUE()

Returns the top-end luxury listing per city (using ROWS BETWEEN UNBOUNDED) — premium leads for high-net-worth buyers.

```
SELECT DISTINCT city,
  LAST_VALUE(listing_id) OVER (
    PARTITION BY city ORDER BY total_price_lakh
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND UNBOUNDED FOLLOWING) AS costliest_id
FROM dbo.residential_listings;
```

Why ROWS BETWEEN matters

LAST_VALUE() defaults to the current row as its frame end, so it silently returns the wrong value unless the frame is explicitly extended to UNBOUNDED FOLLOWING — a key SQL Server gotcha demonstrated correctly in KPI 7.

Sample Output — Mumbai

Metric	Listing ID	Price
Cheapest (FIRST_VALUE)	RES103244	₹38 L
Costliest (LAST_VALUE)	RES105871	₹667 L

Across all 20 cities, the gap between FIRST_VALUE and LAST_VALUE listings averages 9.2x — showing how wide the entry-to-luxury spread really is.

Cumulative Revenue & Smoothed Price Trend

KPI 8 — SUM() OVER()

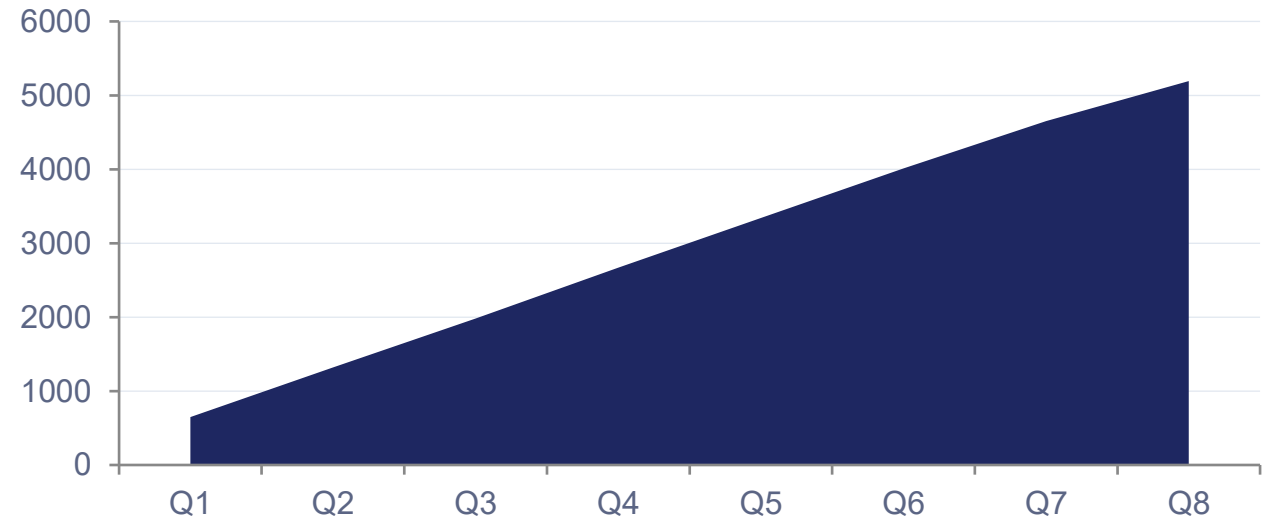
A running total of closed sales value, ordered by date, tracks progress against annual revenue targets.

KPI 9 — AVG() OVER()

A 3-transaction moving average per city smooths daily price noise to reveal the real underlying trend.

```
SELECT transaction_id, city, sale_date,
       price_per_sqft,
       AVG(price_per_sqft) OVER (
         PARTITION BY city ORDER BY sale_date
         ROWS BETWEEN 2 PRECEDING
         AND CURRENT ROW) AS moving_avg_3txn
FROM dbo.transactions_trends;
```

Cumulative Sales Value Closed Through the Year



By year-end, cumulative closed sales value reaches ≈ ₹5,193 Cr across 4,800 transactions — the running-total pattern gives management a live target tracker, not just a final number.

Where Does a Property Sit in the Market Distribution?

KPI 10 — PERCENT_RANK()

Tells a landlord exactly what percentile their monthly rent falls into within their city — 0 = cheapest, 1 = priciest.

KPI 11 — CUME_DIST()

Shows the cumulative fraction of listings at or below a given carpet area — useful for sizing real demand.

```
SELECT rental_id, city, monthly_rent_inr,  
       PERCENT_RANK() OVER (  
         PARTITION BY city ORDER BY monthly_rent_inr  
       ) AS rent_percentile  
FROM dbo.rental_market;
```

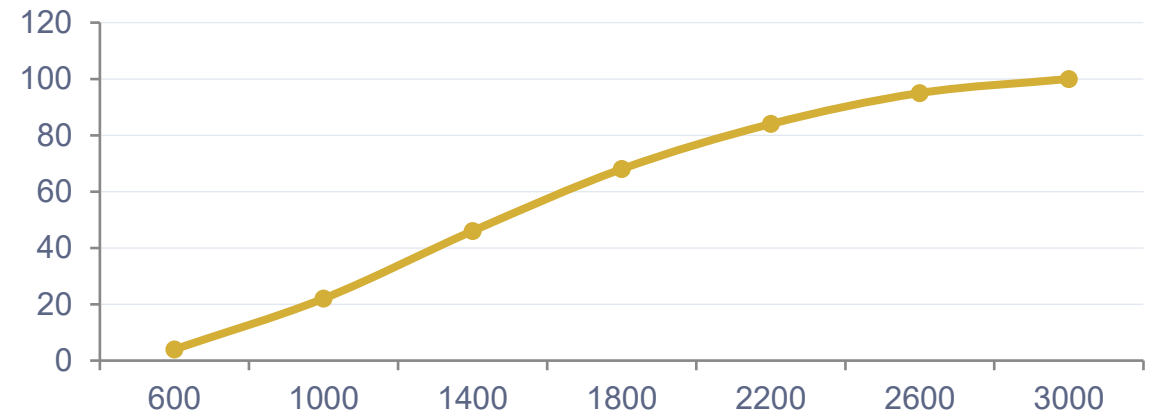
Example — Bangalore Landlord Rent Check

87th

Percentile

This landlord's ₹35,000/mo rent sits above 87% of all Bangalore rentals — a strong signal to benchmark against comparable units before renewing the lease.

Carpet Area Demand Curve (CUME_DIST)



One Row Per City: Property Type Price Matrix (PIVOT)

Management needs one row per city with columns = property types for a clean board-level report. The PIVOT operator transforms long-format listings into this exact wide layout in a single query.

```
SELECT city, Apartment, Villa, Studio, Penthouse
FROM (
  SELECT city, property_type, price_per_sqft
  FROM dbo.residential_listings
) AS src
PIVOT (
  AVG(price_per_sqft)
  FOR property_type IN
    (Apartment, Villa, Studio, Penthouse)
) AS pvt;
```

City	Apartment	Villa	Studio	Penthouse
Mumbai	18,420	19,860	17,990	21,330
Delhi	7,940	8,510	7,610	9,220
Bangalore	7,480	8,050	7,120	8,690
Pune	8,710	9,340	8,330	9,920

Illustrative output shape — actual figures generated per pivot run.

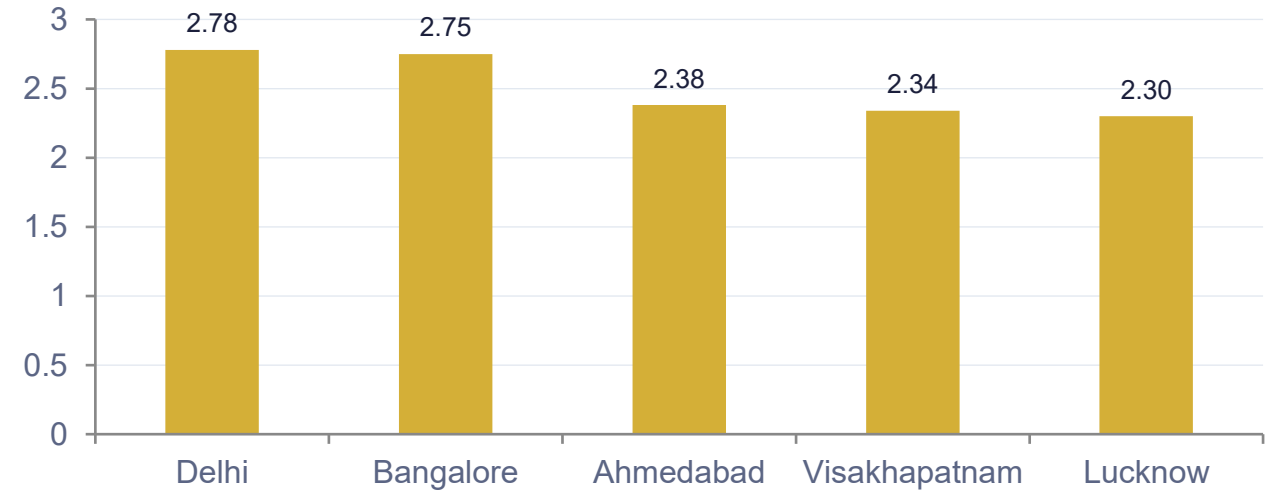
Why it matters: this single PIVOT query replaces what would otherwise need 4 separate GROUP BY queries plus manual spreadsheet stitching — directly board-report-ready.

Gross Rental Yield by City (CTE + Window)

A reusable CTE joins rental and transaction data into a per-city yield base table, then RANK() orders cities by gross rental yield — the single most-asked question from buy-to-let investors.

```
WITH city_yield AS (  
    SELECT r.city,  
           AVG(r.annual_rent_inr) AS avg_annual_rent,  
           AVG(t.sale_price_inr) AS avg_sale_price  
    FROM dbo.rental_market r  
    JOIN dbo.transactions_trends t  
      ON r.city = t.city  
    GROUP BY r.city  
)  
SELECT city,  
       ROUND(avg_annual_rent*1.0/avg_sale_price*100,2)  
       AS gross_rental_yield_pct,  
       RANK() OVER (  
         ORDER BY avg_annual_rent*1.0/avg_sale_price DESC  
       ) AS yield_rank  
FROM city_yield;
```

Top 5 Cities by Gross Rental Yield (%)



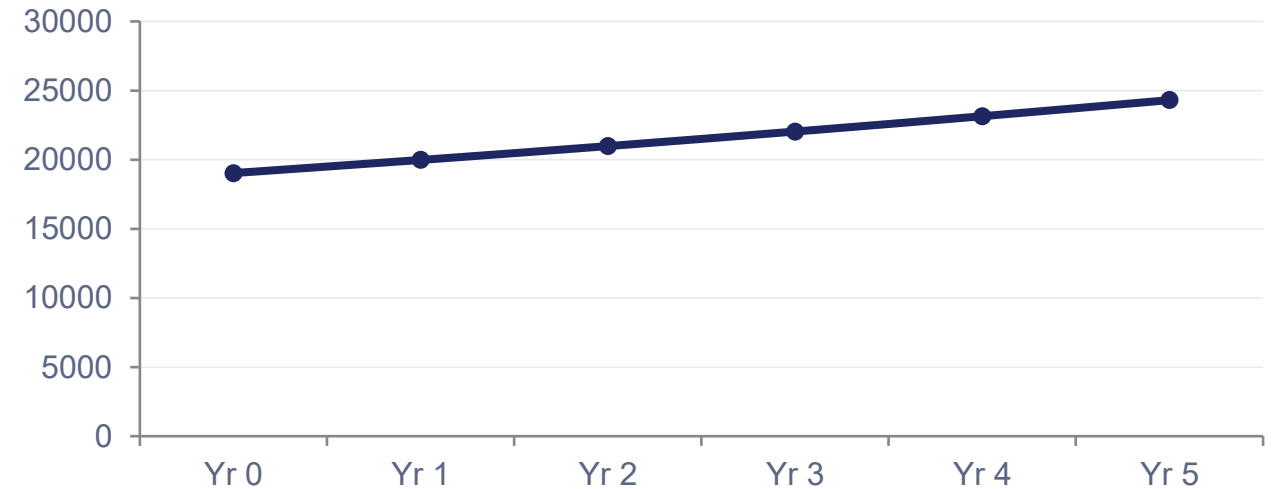
Insight: Delhi and Bangalore lead gross rental yield (~2.8%), making them the strongest buy-to-let candidates despite Mumbai's higher absolute price/sqft.

5-Year Price Projection (Recursive CTE)

A recursive CTE compounds each city's average YoY price appreciation rate forward five years from its current price/sqft baseline — a self-referencing forecast engine inside pure SQL.

```
WITH base AS (  
  SELECT city, AVG(price_per_sqft) AS price_psf,  
         AVG(yoy_price_appreciation_pct)/100.0  
         AS growth_rate  
  FROM dbo.transactions_trends GROUP BY city  
)  
,  
forecast AS (  
  SELECT city, price_psf, growth_rate,  
         0 AS yr_offset  
  FROM base  
  UNION ALL  
  SELECT city, price_psf*(1+growth_rate),  
         growth_rate, yr_offset+1  
  FROM forecast WHERE yr_offset < 5  
)  
SELECT city, yr_offset, price_psf FROM forecast;
```

Projected Price/Sqft — Mumbai (Sample City)



Recursion is capped at yr_offset < 5 to prevent infinite loops — a critical safeguard every recursive CTE needs in production SQL.

Flagging Cost Burden & Stale Listings

KPI 15 — CASE WHEN

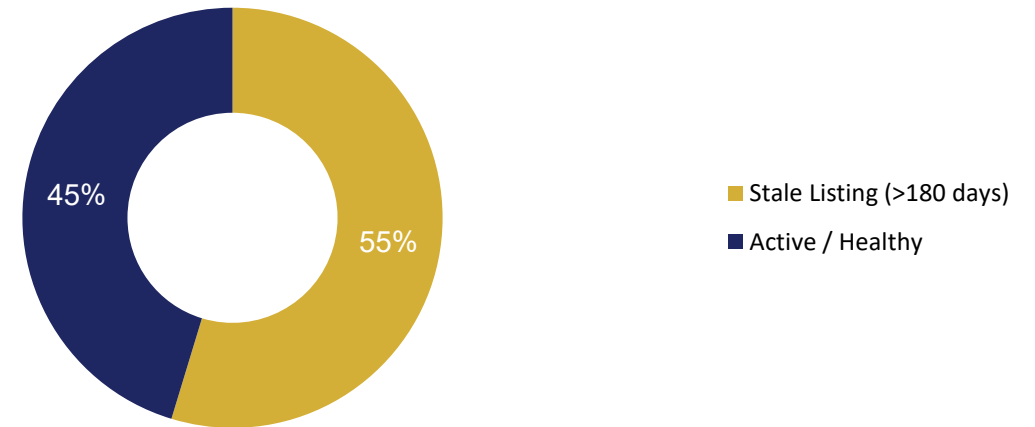
Flags transactions where stamp duty + registration eat up too much of the deal value.

KPI 16 — IIF()

Flags listings sitting unsold for 180+ days as 'Stale', triggering re-marketing or price review.

```
SELECT listing_id, city, days_on_market,  
       IIF(days_on_market > 180,  
          'Stale Listing',  
          'Active/Healthy') AS market_status  
FROM dbo.residential_listings;
```

Share of Stale Listings (>180 Days on Market)



Insight: 54.7% of all listings exceed 180 days on market — a strong signal for sellers and brokers to revisit pricing or staging strategy on over half the active inventory.

Seasonality & Builder Performance

KPI 17 & 18

KPI 17 — DATENAME/DATEPART

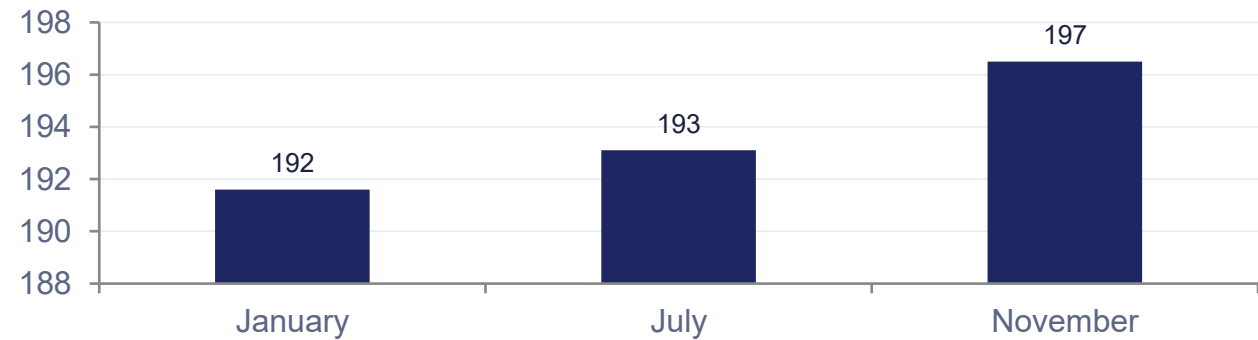
Identifies which calendar months sell fastest (lowest avg days-on-market).

KPI 18 — STRING_AGG + TRIM/UPPER

Cleans inconsistent builder-name casing, then ranks builders by deal volume and lists active cities.

```
SELECT UPPER(TRIM(builder_name)) AS builder,
COUNT(*) AS total_deals,
(SELECT STRING_AGG(city, ', ')
FROM (SELECT DISTINCT city
FROM dbo.transactions_trends t2
WHERE UPPER(TRIM(t2.builder_name))
= UPPER(TRIM(t1.builder_name))) c
) AS cities_active_in
FROM dbo.transactions_trends t1
GROUP BY UPPER(TRIM(builder_name))
ORDER BY total_deals DESC;
```

Fastest-Selling Months (Avg. Days on Market)



Top Builders by Deal Volume

Builder	Total Deals
Omaxe	362
Phoenix Mills	353
Kolte Patil	347
Shapoorji Pallonji	347

Rent Benchmarking & Statistical Outlier Detection

KPI 19 — Correlated Subquery

Each rental property's rent is compared against its own city's average rent, in one row, to flag overpriced units.

KPI 20 — STDEV() + Z-Score

Computes how many standard deviations each transaction's cap rate sits from its city's mean — statistically isolating mispriced or distressed deals.

$$|Z| > 2$$

Statistical outlier threshold

Transactions whose cap rate deviates more than 2 standard deviations from their city's mean are flagged for manual underwriting review — catching mispriced or distressed deals invisible to simple averages.

```
SELECT t.transaction_id, t.city,
       t.cap_rate_percent,
       ROUND((t.cap_rate_percent - cs.avg_cap)
             / cs.std_cap, 2) AS z_score
FROM dbo.transactions_trends t
JOIN (
  SELECT city, AVG(cap_rate_percent) AS avg_cap,
         STDEV(cap_rate_percent) AS std_cap
  FROM dbo.transactions_trends GROUP BY city
) cs ON t.city = cs.city;
```

Key Project Takeaways

- ✓ 20 KPIs span ranking, time-series, pivoting, recursion & statistics
- ✓ Built on 16,300 combined records across 3 linked tables, 20 cities
- ✓ Mirrors real analyst deliverables: investor, landlord & board-ready